

CompressIQ: Optimal Gradient Compression for Faster Distributed ML Training

Project Abstract

Course: Introduction to Optimization

Team Members:

- Surendra Kumar Chandrasekaran - 122270114
- Gowtham Senthil - 121936868
- Aniruddhan Narasimhan – 121955555

Introduction:

Every time a distributed ML model takes a training step, all GPUs in the cluster must agree on the same gradient update before moving forward. They do this through a process called All-Reduce, where each GPU sends its locally computed gradients around a ring until every worker holds the same averaged result. It works well in theory, but in practice it is slow. For large models like BERT or ResNet, the time spent transmitting gradients can easily exceed the time spent on actual computation.

The standard fix is gradient compression: instead of sending the full gradient, each worker sends a compressed version using techniques like Top-K sparsification, where only the largest gradient values are transmitted. This cuts communication time significantly. The catch is that compression introduces approximation error, and too much of it causes the model to learn incorrectly.

Most systems today apply the same compression ratio to every worker regardless of their network speed or gradient size. This is a missed opportunity. A worker on a slow link should compress more aggressively, while a worker on a fast link can afford to transmit at higher fidelity. CompressIQ treats this as what it actually is: an optimization problem.

Optimization Formulation:

We model a ring All-Reduce setup with N workers. Each worker i has a gradient of size G_i bits and communicates over a link with bandwidth B_i bits per second.

The decision variable for each worker is its compression ratio r_i , which ranges from a minimum compression floor r_{min} up to 1 (no compression at all).

Decision Variable:

$r_i \in [r_{min}, 1]$ for each worker i (fraction of gradient retained after compression)

Objective Function:

$$\min \max_i \left\{ \frac{r_i \cdot G_i}{B_i} \right\}$$

This minimizes the bottleneck communication time across all workers. Because the ring proceeds at the speed of its slowest worker, minimizing the maximum transmission time is what actually speeds up training.

Constraints:

1. Accuracy loss Bound:

$$\sum_i \alpha_i (1 - r_i)^2 \leq \varepsilon$$

where α_i = accuracy sensitivity of worker i 's gradients

ε = maximum allowable accuracy loss over the cluster

2. Compression Bounds:

$$r_{min} \leq r_i \leq 1 \quad \forall i$$

The objective is a maximum of linear functions, which is convex. The accuracy constraint is a sum of convex quadratic terms, also convex. The full problem is therefore a convex program and can be solved efficiently using cvxpy with the ECOS or SCS solver. No integer variables are needed

Proposed Methodology

The optimization core is implemented in approximately 30 lines of cvxpy. The synthetic environment models 8 workers in a ring, with gradient sizes based on published parameter counts of BERT-base (110M parameters) and ResNet-50 (25M parameters), and bandwidth values ranging from 1 to 10 Gbps across intra-rack and inter-rack links. The accuracy loss model follows the quadratic approximation from Top-K sparsification literature, with coefficients α_i set per worker.

CompressIQ is evaluated against three baselines: no compression, uniform compression (fixed ratio across all workers), and greedy compression (each worker compresses independently without coordination). Sensitivity analysis will be conducted across varying accuracy budgets and cluster sizes, with a Pareto frontier plot illustrating the tradeoff between communication speedup and accuracy loss.

Goals and Expected Outcomes

The central claim of CompressIQ is simple: per-worker compression ratios, optimized through a convex formulation, will outperform uniform compression for any given accuracy budget.

Deliverables:

1. Reproducible Python implementation on GitHub
2. Pareto frontier and per-worker compression visualizations
3. Baseline comparison across multiple accuracy budgets and cluster sizes